



The Dynamic Hatchery 1.0.0

Malloc, Calloc, Realloc, Free

▼ Guidelines

Introduction

Lore

You have joined the Dragon Academy, as a new apprentice. Your mission as an aspiring hatchery worker is to learn how to hatch dragons, care for their nests and manage them wisely. Dragons grow in dynamic nests which have the advantage of being controllable during their growth -- you might: request space in the hatchery for them, grow nests if space is insufficient and release them safely when the time is right.

Indication

The purpose of this practical is to give you some basic knowledge about dynamic memory allocation. You are going to learn how to allocate memory at runtime, manage it safely, and free it when it is no longer needed.

Warning(s)

In this practical you can only use functions from the standard library if specifically asked or suggested to. Unless this function is `malloc`, `calloc`, `realloc` or `free`, in that case you may use them freely except when the subject tells you to use one over the others.

The libraries you might need for this practical are `stdlib.h` and `string.h`.

It is good practice to check that the functions `malloc`, `calloc` and `realloc` successfully ran, in the opposite case they will return `NULL`.

▲ Fundamentals - hatching, growing, releasing, managing...

Dive into the exciting world of dragon hatching.

basics.c

Lore

Your first job is to learn the basics of hatching for this you are given a few tools by Gobber to make dynamically hatching easier...

Hint(s)

Check [man malloc](#).

▲ Hatch a dragon

You are first taught to bring your first dragon to life using a `malloc`. The value of the dragon corresponds to its age and should be 0.

Indication

If `malloc` fails to allocate, return `NULL`.

Prototype(s)

```
int *hatch();
```

Code example

```
int *hatchling = hatch();
printf("%d\n", *hatchling); // 0

free(hatchling);
```

▲ Release a dragon

Set a dragon free once it has reached the right age. If not, do nothing. In both cases return the difference between its age and the release age.

A dragon has reached the right age when its age is greater or equal than the release age.

Prototype(s)

```
int release(int *dragon, int release_age);
```

Code example

```
int *toothless = hatch();
*toothless = 15;
printf("%d\n", release(toothless, 10)); // 5 (was released)
int *stormfly = hatch();
*stormfly = 10;
printf("%d\n", release(stormfly, 11)); // -1 (was not released)
free(stormfly);
```

▲ Hatch a nest

To care for multiple dragons at once you must create a nest using a `calloc`.

Hint(s)

`calloc` allocates a contiguous block of memory and returns the pointer to the beginning of the array.

Prototype(s)

```
int *hatch_nest(size_t amount);
```

Code example

```
int *nest = hatch_nest(10);
for (size_t i = 0; i < 10; ++i) {
    printf("%d, ", nest[i]);
}
printf("\n"); // 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
free(nest);
```

▲ Grow a nest

Expand the nest when new eggs arrive using a `realloc`.

Make sure that the new eggs are of age 0.

Hint(s)

You may use [memset](#).

Indication

If `realloc` fails to allocate the size, return `NULL`. `new_size` must be greater than `old_size`. If not, return `NULL`.

Prototype(s)

```
int *grow_nest(int *nest, size_t old_size, size_t new_size);
```

Code example

```
int *nest = hatch_nest(12);
int *new_nest = grow_nest(nest, 12, 14);
for (size_t i = 0; i < 14; ++i) {
    printf("%d, ", new_nest[i]);
}
printf("\n"); // 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
free(new_nest);
```

▲ Release dragons from a nest

Send the elder dragons to the sky, while keeping the youngs.

The dragons should be freed if they are above the release age, and kept in the output nest if not. Return the size of the output nest.

There might be no dragon left in the nest, in that case **make sure** that the `out_nest` is freed or wasn't allocated.

Hint(s)

`out_nest` and `out_*` parameters alike will be ways for you to return multiple objects via arguments. Here it represents the resulting array of dragons that are not of age.

Indications

Free the original nest, and allocate a new one.

We assume that the nest passed is **always** allocated on the heap.

In the event `malloc` fails return a size of `0` and leave `*out_nest` to `NULL`.

Prototype(s)

```
size_t release_nest(const int *nest, size_t size,
                   int release_age, int **out_nest);
```

Code example

```
int *nest = hatch_nest(10);
int *out_nest;

for (size_t i = 0; i < 10; ++i) {
    nest[i] = i % 5;
}
size_t size = release_nest(nest, 10, 4, &out_nest);

for (size_t i = 0; i < size; ++i) {
    printf("%d, ", out_nest[i]);
}
printf("\n"); // 0, 1, 2, 3, 0, 1, 2, 3,
free(out_nest);
```

nest_management.c

Lore

You are getting pretty good at this and your prowess has caught the attention of Fishlegs, who is hungry for new knowledge. He wants you to manage the nests and get insightful information on the hatchlings.

▲ To string

The first information that you can gather is the ages of the dragons from each nest, the number of nests is given by `size`. However you want to write the ages automatically to scrolls.

Indications

The output string **has to** be each age separated by spaces and end with a space and a `'\n'`, dragons cannot age passed 999.

You should return a string allocated on the heap.

You should put a `\n` at the end.

Warning(s)

Do not forget the null terminator `\0` at the end of the string.

Prototype(s)

```
char *to_string(const int *nest, size_t size);
```

Code example

```
int *nest = hatch_nest(10);
for (int i = 0; i < 10; ++i) {
    nest[i] = i + 1;
}
char *output = to_string(nest, 10);
printf("%s\n", output); // 1 2 3 4 5 6 7 8 9 10
free(output);
```

▲ Mirror nests

Nests can be mirrored which almost doubles them in eggs all using a simple algorithm. To make it you simply need to make them palindromic without repeating the last number of the original sequence.

Example

For example this nest: 1 2 3 9 turns into this after transformation: 1 2 3 9 3 2 1.

Indication

You must use `realloc` on `nest` for this exercise.

Prototype(s)

```
int *mirror_nest(const int *nest, size_t size);
```

Code example

```
int *nest = hatch_nest(5);
for (int i = 0; i < 5; ++i) {
    nest[i] = i * 2;
}
int *mirrored_nest = mirror_nest(nest, 5);
printf("%s\n", to_string(mirrored_nest, 5 * 2 - 1)); // 0 2 4 6 8 6 4 2 0
```



▲ Mirror nests 2

Same thing as last exercise but simply modify the pointer to the pointer to the nest, such that it points to the reallocated memory.

Prototype(s)

```
void mirror_nest_2(const int **nest, size_t size);
```

Code example

```
int *nest = hatch_nest(5);
for (size_t i = 0; i < 5; ++i) {
    nest[i] = i * 2;
}
mirror_nest_2(&nest, 5);
printf("%s\n", to_string(nest, 5 * 2 - 1)); // 0 2 4 6 8 6 4 2 0
```

▲ Separate hatchlings by ages

Split the dragons into multiple nests depending on their age groups. These age groups are separated by thresholds.

For each threshold there's a group lower and greater or equal than threshold (*not exclusive*). Which makes the number of groups `len(thresholds) + 1`. **It should preserve order.**

Indication

Both `out_nests` and `out_sizes` **should be** allocated on the heap.

`nest` **shouldn't** be freed.

Prototype(s)

```
void separate(const int *nest, size_t size, int *thresholds,
              size_t thresholds_size, int ***out_nests, size_t **out_size
```



Code example

```
int *nest = hatch_nest(9);
int thresholds[] = { 2, 4 };
for (size_t i = 0; i < 9; ++i) {
    nest[i] = (i * 3) % 7;
}
printf("%s\n", to_string(nest, 9)); // 0 3 6 2 5 1 4 0 3
int **out_nests;
size_t *out_sizes;
separate(nest, 9, thresholds, 2, &out_nests, &out_sizes);
for (size_t i = 0; i < 3; ++i) {
    printf("| %s ", to_string(out_nests[i], out_sizes[i]));
    free(out_nests[i]);
}
printf("| \n"); // | 0 1 0 | 3 2 3 | 6 5 4 |
free(nest);
free(out_nests);
free(out_sizes);
```

 With kinds

We now want to construct nests that are capable of informing the kind of each dragons. For this we allocate a new nest in the format `age(dragon-kind)`
`nest` contains the ages of the dragons and `kinds` contains their kinds.

Indication

You must return a heap-allocated array of heap-allocated strings.
You shouldn't free ages.
You can use `strlen` and `strcpy` from `string.h`.

Prototype(s)

```
char **with_kinds(int *ages, const char **kinds, size_t size);
```

Code example

```
int *nest = hatch_nest(4);
char *kinds[] = {
    "Night Fury",
    "Deadly Nadder",
    "Monstrous Nightmare",
    "Gronkle",
};
for (int i = 0; i < 4; ++i) {
    nest[i] = i * 5 + 5;
}

char **nest_with_kinds = with_kinds(nest, kinds, 4);
for (size_t i = 0; i < 4; ++i) {
    printf("%s ", nest_with_kinds[i]);
}
printf("\n"); // 5(Night Fury) 10(Deadly Nadder) 15(Monstrous Nightmare)
```

```
free(nest);  
free(nest_with_kinds);
```

▲ Single kinds and get ages back

Now we want to be able to read the ages and kinds back.

The ages goes in `*out_ages` and the kinds in `*out_kinds`.

Indication

`nest` is already heap-allocated and **you must** free it.

You can use `strlen` and `strcpy` from `string.h`.

Prototype(s)

```
void single_kinds(char **nest, size_t size, int **out_ages,  
                  char ***out_kinds);
```

Code example

```
int *nest = hatch_nest(4);  
char *kinds[] = {  
    "Night Fury",  
    "Deadly Nadder",  
    "Monstrous Nightmare",  
    "Gronkle",  
};  
for (int i = 0; i < 4; ++i) {  
    nest[i] = i * 5 + 5;  
}  
char **nest_with_kinds = with_kinds(nest, (const char **)kinds, 4);  
free(nest);
```

```
int *out_ages;
char **out_kinds;
single_kinds(nest_with_kinds, 4, &out_ages, &out_kinds);
printf("%s\n", to_string(out_ages, 4)); // 5 10 15 20
for (size_t i = 0; i < 4; ++i) {
    printf("%s, ", out_kinds[i]);
}
printf("\n"); // Night Fury, Deadly Nadder, Monstrous Nightmare, Gronkle,
free(out_ages);

for (size_t i = 0; i < 4; ++i) {
    free(out_kinds[i]);
}
free(out_kinds);
```

▲ Single kinds and get ages back reused

Same thing as before but we extract the kinds to store them in `nest`, effectively reusing memory.

Indication

`nest` is already heap-allocated and **you must** reuse it.

Prototype(s)

```
void single_kinds_reused(char **nest, size_t size, int ***out_ages);
```

Code example

```
int *nest = hatch_nest(4);
char *kinds[] = {
    "Night Fury",
    "Deadly Nadder",
    "Monstrous Nightmare",
```

```
"Gronkle",
};
for (int i = 0; i < 4; ++i) {
    nest[i] = i * 5 + 5;
}
char **nest_with_kinds = with_kinds(nest, (const char **)kinds, 4);
free(nest);

int *out_ages;
single_kinds_reused(nest_with_kinds, 4, &out_ages);
printf("%s\n", to_string(out_ages, 4)); // 5 10 15 20
for (size_t i = 0; i < 4; ++i) {
    printf("%s, ", nest_with_kinds[i]);
}
printf("\n"); // Night Fury, Deadly Nadder, Monstrous Nightmare, Gronkle,
free(out_ages);
free(nest_with_kinds);
```

▲ Proficiencies - sanctuaries

As you are evolving in the world of hatchery another project is brought to your attention by Valka, it seems that to get to the next level you need to be managing sanctuaries. The latter suggests that the more natural way to store nests is in shared spaces where all the dragons can be in sync with each other.

nest_allocator.c

Lore

Let's start by building the nest allocator.

Info

See [region based memory management](#).

Info

This is not standard C practice, but demonstrates memory pools often used in games/databases.

▲ Utilities

Here is the header you will be using, it contains a `struct` and an `enum` to help you make the following exercises.

Reminder

In order to access elements in structs you can use the operator `..`
If that struct is stored in a pointer then you may use `->`.
You can learn more about enums [here](#).

Header

```
#ifndef NEST_ALLOCATOR_H
#define NEST_ALLOCATOR_H

#include <stdlib.h>

struct nest_allocator {
    void *data;
    char *uri;
    size_t capacity;
    size_t offset;
};

enum xsd_type {
    XSD_any_uri,
    XSD_integer,
    XSD_float,
    XSD_string,
    XSD_dateTime,
```

```
XSD_boolean,
XSD_double,
XSD_decimal,
#   define XSD_sizeof(ty) xsd_sizes[ty]
};

extern size_t xsd_sizes[];
/* In the c file
size_t xsd_sizes[] = {
    [XSD_integer ] = sizeof(int),
    [XSD_float    ] = sizeof(float),
    [XSD_string   ] = sizeof(char *),
    [XSD_dateTime] = sizeof(char *),
    [XSD_boolean  ] = sizeof(int),
    [XSD_double   ] = sizeof(double),
    [XSD_decimal  ] = sizeof(long double),
};
*/

struct nest_allocator *nest_init(size_t capacity, char *uri);
void *nest_hatch(struct nest_allocator *nest, size_t alloc_size);
void nest_reset(struct nest_allocator *nest);
void nest_release_all(struct nest_allocator *nest);

#endif //NEST_ALLOCATOR_H
```

▲ Initialize

Initialize the nest (with the type `struct nest_allocator`) by allocating itself and the revolving data (with `capacity` elements) to the heap. Also update its capacity. As for the uri field, simply assign it to the parameter of the same name. The function returns the allocated `struct nest_allocator`.

Indication

Data should be one big contiguous block of memory.

`offset` should be initialize to `0`.

As it's a big block of memory **you must** check if malloc was successfull and return `NULL` if not.

Prototype(s)

```
struct nest_allocator *nest_init(size_t capacity, char *uri);
```

▲ Allocate

Allocate memory in the nest by adding to the offset and returning the pointer to the offsetted memory.

Warning(s)

Be aware that the allocation might exceed the capacity.

Prototype(s)

```
void *nest_hatch(struct nest_allocator *nest, size_t alloc_size);
```

▲ Reset

Reset all allocations by resetting the offset.

Prototype(s)

```
void nest_reset(struct nest_allocator *nest);
```

 Free

Release all the dragons from the allocator.

Warning(s)

Don't forget to free the allocator.

Prototype(s)

```
void nest_release_all(struct nest_allocator *nest);
```

sanctuary_graph.c

Lore

Now that we have sorted the creation of the nest allocator. We want to display the sanctuary in the form of a beautiful graph.

Info

We will be using the [turtle language](#) and rdf graphs.

Warning(s)

In all these exercises you will allocate memory through the `nest_allocator` struct and not free any allocations in the functions directly, you will be asked to do it later using

the nest allocator. In each exercise be aware that the capacity for the nest might be exceeded, **you should** return **NULL** in those cases.

▲ To URI

Convert a name to a URI.

Hint(s)

In turtle URIs are represented like this `<http://example.com#{name}>`. As shown in the URI we will be using `example.com` as a domain name.

Prototype(s)

```
char *to_uri(struct nest_allocator *nest, const char *name);
```

▲ To literal

Convert the dragon's information into a turtle literal.

Hint(s)

Typed literals in turtle are in the form `"{data}"^^xsd:{xsd-type}`. Make sure that each types are handled correctly and cast the void pointer to the pointer type you need to use (ex: `*(int *)data`).

Hint(s)

You may use [sprintf](#) for conversions.

In order to know the size of the formatted string to allocate, **you may** use `snprintf(NULL, 0, {formatted-string}, {va-args})`.

Hint(s)

See [xsd types](#).

Indication

In the case of floats **you must** print them with 1, 3 and 5 digits precision for floats, doubles and long doubles respectively.

Prototype(s)

```
char *to_literal(struct nest_allocator *nest,
                const void *data, enum xsd_type type);
```

▲ Output a triple

Output a single relation using a rdf triple.

Hint(s)

RDF triples are in the format `{subj-uri} {rel-uri} {obj} .\n` (*obj can be a uri or a literal*).

Prototype(s)

```
void output_triple(const char *subj, const char *rel, const char *objs);
```



▲ To nest of URIs

Create an array of URIs from the array of names and allocate it in the nest. Also output a RDF triple that shows that each dragon's URIs belong to the nest using the relation `<http://example.com#belongsTo>`.

Hint(s)

You may use `to_uri` and `output_triple`.

Prototype(s)

```
char **to_uri_nest(struct nest_allocator *nest,  
                  char **names, size_t size);
```

▲ To nest of Literals

Create an array of literals from the array of generic data and allocate it in the nest. Handle the size correctly using `XSD_sizeof`.

Hint(s)

You may use `nest_hatch` and `to_literal`.

Prototype(s)

```
char **to_literal_nest(struct nest_allocator *nest,  
                      void *literals, enum xsd_type type, size_t size);
```

▲ Output triples

Output multiple relations using RDF triples.

Hint(s)

You may use `output_triple`.

Prototype(s)

```
void output_triples(const char **subjs, const char *rel, const char **obj  
                  size_t size);
```

main.c

▲ Makefile

Code example(s)

```
CC := gcc  
CFLAGS := -Wall -Wextra -fsanitize=address  
TARGET := sanctuary
```

```
SRC := main.c sanctuary_graph.c nest_allocator.c
```

```
$(TARGET): $(SRC)
    $(CC) $(CFLAGS) -o $@ $(SRC)
```

```
.PHONY: clean
```

```
clean:
    rm -f $(TARGET)
```

▲ Construct the sanctuary

It is now time to construct the sanctuary by replicating the following turtle code.

Indication

You should add `@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .`

Turtle

```
<http://example.com#Toothless> <http://example.com#belongsTo> <http://example.com#Stormfly> <http://example.com#belongsTo> <http://example.com#Hookfang> <http://example.com#belongsTo> <http://example.com#Meatlug> <http://example.com#belongsTo> <http://example.com#Stormfly>
```

```
<http://example.com#Toothless> <http://example.com#name> "Toothless"^^xsd:string
<http://example.com#Stormfly> <http://example.com#name> "Stormfly"^^xsd:string
<http://example.com#Hookfang> <http://example.com#name> "Hookfang"^^xsd:string
<http://example.com#Meatlug> <http://example.com#name> "Meatlug"^^xsd:string
```

```
<http://example.com#Toothless> <http://example.com#born> "2010-05-26T00:00:00"^^xsd:string
<http://example.com#Stormfly> <http://example.com#born> "2011-02-26T00:00:00"^^xsd:string
<http://example.com#Hookfang> <http://example.com#born> "2009-03-26T00:00:00"^^xsd:string
<http://example.com#Meatlug> <http://example.com#born> "2011-07-26T00:00:00"^^xsd:string
```

```
<http://example.com#Toothless> <http://example.com#age> "15"^^xsd:integer
<http://example.com#Stormfly> <http://example.com#age> "14"^^xsd:integer
<http://example.com#Hookfang> <http://example.com#age> "16"^^xsd:integer
<http://example.com#Meatlug> <http://example.com#age> "14"^^xsd:integer .
```

```
<http://example.com#Toothless> <http://example.com#weighs> "1510.5"^^xsd:
<http://example.com#Stormfly> <http://example.com#weighs> "1234.5"^^xsd:f
<http://example.com#Hookfang> <http://example.com#weighs> "2000.1"^^xsd:f
<http://example.com#Meatlug> <http://example.com#weighs> "2500.0"^^xsd:fl
```

```
<http://example.com#Toothless> <http://example.com#canFly> "true"^^xsd:bo
<http://example.com#Stormfly> <http://example.com#canFly> "true"^^xsd:boo
<http://example.com#Hookfang> <http://example.com#canFly> "true"^^xsd:boo
<http://example.com#Meatlug> <http://example.com#canFly> "true"^^xsd:bool
```

```
<http://example.com#Barf> <http://example.com#belongsTo> <http://example.
<http://example.com#Belch> <http://example.com#belongsTo> <http://example
<http://example.com#Sharpshot> <http://example.com#belongsTo> <http://exa
```

```
<http://example.com#Barf> <http://example.com#name> "Barf"^^xsd:string .
<http://example.com#Belch> <http://example.com#name> "Belch"^^xsd:string
<http://example.com#Sharpshot> <http://example.com#name> "Sharpshot"^^xsd
```

```
<http://example.com#Barf> <http://example.com#born> "2022-06-13T00:00:00"
<http://example.com#Belch> <http://example.com#born> "2022-05-13T00:00:00
<http://example.com#Sharpshot> <http://example.com#born> "2023-07-13T00:0
```

```
<http://example.com#Barf> <http://example.com#age> "3"^^xsd:integer .
<http://example.com#Belch> <http://example.com#age> "3"^^xsd:integer .
<http://example.com#Sharpshot> <http://example.com#age> "2"^^xsd:integer
```

```
<http://example.com#Barf> <http://example.com#weighs> "322.2"^^xsd:float
<http://example.com#Belch> <http://example.com#weighs> "304.4"^^xsd:float
<http://example.com#Sharpshot> <http://example.com#weighs> "242.4"^^xsd:f
```

```
<http://example.com#Barf> <http://example.com#canFly> "true"^^xsd:boolean
<http://example.com#Belch> <http://example.com#canFly> "true"^^xsd:boolea
<http://example.com#Sharpshot> <http://example.com#canFly> "true"^^xsd:bo
```

```
<http://example.com#Frostclaw> <http://example.com#belongsTo> <http://exa
<http://example.com#Inferno> <http://example.com#belongsTo> <http://examp
<http://example.com#Zephyrwing> <http://example.com#belongsTo> <http://ex
```

```
<http://example.com#Frostclaw> <http://example.com#name> "Frostclaw"^^xsd
<http://example.com#Inferno> <http://example.com#name> "Inferno"^^xsd:str
<http://example.com#Zephyrwing> <http://example.com#name> "Zephyrwing"^^x
```

```
<http://example.com#Frostclaw> <http://example.com#born> "2020-01-01T00:0
<http://example.com#Inferno> <http://example.com#born> "2019-05-05T00:00:
<http://example.com#Zephyrwing> <http://example.com#born> "2018-12-25T00:
```

```
<http://example.com#Frostclaw> <http://example.com#age> "5"^^xsd:integer
<http://example.com#Inferno> <http://example.com#age> "6"^^xsd:integer .
<http://example.com#Zephyrwing> <http://example.com#age> "7"^^xsd:integer
```

```
<http://example.com#Frostclaw> <http://example.com#weighs> "1844.2"^^xsd:
<http://example.com#Inferno> <http://example.com#weighs> "2222.2"^^xsd:fl
<http://example.com#Zephyrwing> <http://example.com#weighs> "1445.5"^^xsd

<http://example.com#Frostclaw> <http://example.com#canFly> "true"^^xsd:bo
<http://example.com#Inferno> <http://example.com#canFly> "true"^^xsd:bool
<http://example.com#Zephyrwing> <http://example.com#canFly> "true"^^xsd:b
```

Prototype(s)

```
int main();
```

▲ See my sanctuary

Copy the content of the output you've created then put it into [this site](#) to visualize your creation.

The command given below will copy the output of your program.

Copy command

```
./sanctuary | xsel --clipboard --input
```

This page and all subpages are for internal use at EPITA only.

The use of this document must abide by the following rules:

- You are seeing it from the EPITA' Gitlab pages.
- This page and all subpages are strictly personal and must not be passed onto someone else.
- Non-compliance with these rules can lead to severe sanctions.

Copyright © 2025-2026 - EPITA